Laporan Praktikum: MLflow Experiment Tracking & CI/CD/CT

Mata Kuliah: Machine Learning Operations (MLOps) • **Program Studi Sains Data ITERA**

> **Tugas Praktikum:** Mencoba kode eksperimen MLflow end-to-end menggunakan studi kasus klasifikasi medis (Breast Cancer Wisconsin), mencatat parameter, metrik CV, artefak, serta mengintegrasikannya dengan Model Registry dan Quality Gate otomatis.

1. Persiapan Lingkungan & Server MLflow

Sebelum menjalankan skrip eksperimen, pastikan library pendukung terinstal dan MLflow Tracking Server lokal aktif di latar belakang (menggunakan SQLite sebagai backend store dan folder lokal untuk artifact store).

```
# 1. Instalasi Library yang Dibutuhkan
pip install mlflow scikit-learn pandas matplotlib requests

# 2. Menjalankan MLflow Tracking Server di Terminal 1
mlflow server --backend-store-uri sqlite:///mlflow.db \
              --default-artifact-root ./mlartifacts --host 127.0.0.1 --port 5000
```

2. Arsitektur Pelacakan & Desain Kontrol

Eksperimen dirancang dengan kontrol ketat: dataset dibagi dengan *Stratified K-Fold* (5 folds, seed 42) untuk mengevaluasi model secara objektif. Metrik utama penentu kualitas adalah **Recall** (meminimalkan tumor ganas yang terlewat) dengan syarat batas (*satisficing metric*) **Precision ≥ 0.90**.

3. Kode Implementasi Eksperimen (Python)

Berikut adalah skrip lengkap implementasi tracking, training, evaluasi test set sekali, dan automated quality gate:

```
import os, hashlib, mlflow, mlflow.data, pandas as pd
from sklearn.datasets import load_breast_cancer
from sklearn.model_selection import train_test_split, StratifiedKFold, cross_validate, cross_val_pred
from sklearn.preprocessing import StandardScaler
from sklearn.pipeline import Pipeline
from sklearn.svm import SVC
from sklearn.metrics import recall_score, precision_score, f1_score, roc_auc_score, ConfusionMatrixDi
from mlflow.models import infer_signature
from mlflow import MlflowClient

# Konfigurasi MLflow Server
mlflow.set_tracking_uri("http://localhost:5000")
mlflow.set_experiment("bc-malignant-detection")

# L1: Data Preparation & Versioning Hash
df = load_breast_cancer(as_frame=True).frame
df["target"] = 1 - df["target"]  # 1 = malignant
data_md5 = hashlib.md5(pd.util.hash_pandas_object(df, index=True).values).hexdigest()

train_df, test_df = train_test_split(df, test_size=0.2, stratify=df["target"], random_state=42)
X_train, y_train = train_df.drop(columns="target"), train_df["target"]
X_test, y_test = test_df.drop(columns="target"), test_df["target"]
train_ds = mlflow.data.from_pandas(train_df, source="data/train.csv", name="bc-train", targets="targe
COMMON_TAGS = {"data_md5": data_md5, "split_seed": "42", "dataset": "breast-cancer"}

# Fungsi Logging Eksperimen Terstandar
CV, SCORING = StratifiedKFold(n_splits=5, shuffle=True, random_state=42), ["recall", "precision", "f1

def run_experiment(run_name, pipe, tags=None, note=""):
    with mlflow.start_run(run_name=run_name) as run:
        mlflow.set_tags({**COMMON_TAGS, "candidate": "true", **(tags or {})})
        mlflow.set_tag("mlflow.note.content", note)
        mlflow.log_input(train_ds, context="training")
        mlflow.log_param("pipeline_steps", " -> ".join(pipe.named_steps))
        mlflow.log_params({f"clf_{k}": v for k, v in pipe[-1].get_params().items()})

        cv = cross_validate(pipe, X_train, y_train, cv=CV, scoring=SCORING)
        for m in SCORING:
            mlflow.log_metric(f"cv_{m}_mean", cv[f"test_{m}"].mean())
            mlflow.log_metric(f"cv_{m}_std", cv[f"test_{m}"].std())

        pipe.fit(X_train, y_train)
        sig = infer_signature(X_train, pipe.predict(X_train))
        info = mlflow.sklearn.log_model(pipe, name="model", signature=sig, input_example=X_train.head
        mlflow.set_tag("model_uri", info.model_uri)
        return run.info.run_id

# Eksekusi Model SVC Terintonasi (Tuning C=10)
pipe_svc = Pipeline([("scaler", StandardScaler()), ("clf", SVC(C=10, kernel="rbf", random_state=42))]
best_run_id = run_experiment("10-svc-tuned", pipe_svc, tags={"stage": "tuning", "model_family": "svc"
```

4. Evaluasi Test Set & Integrasi Quality Gate (Lanjutan)

```
# L6: Evaluasi Final pada Test Set (Hanya Sekali)
runs_df = mlflow.search_runs(experiment_names=["bc-malignant-detection"], filter_string=f"run_id = '{
model_uri = runs_df.iloc[0]["tags.model_uri"]
loaded_model = mlflow.sklearn.load_model(model_uri)
y_pred = loaded_model.predict(X_test)

test_metrics = {
    "test_recall": recall_score(y_test, y_pred),
    "test_precision": precision_score(y_test, y_pred),
    "test_f1": f1_score(y_test, y_pred),
    "test_roc_auc": roc_auc_score(y_test, loaded_model.decision_function(X_test))
}

with mlflow.start_run(run_id=best_run_id):
    mlflow.log_metrics(test_metrics)
    mlflow.set_tag("test_evaluated", "true")

# L7: Model Registry & Quality Gate Otomatis
client, MODEL_NAME = MlflowClient(), "bc-malignant-classifier"
mv = mlflow.register_model(model_uri, MODEL_NAME)
client.set_registered_model_alias(MODEL_NAME, "challenger", mv.version)

if test_metrics["test_precision"] >= 0.90:
    client.set_registered_model_alias(MODEL_NAME, "champion", mv.version)
    print(f"[SUCCESS] Model v{mv.version} dipromosikan menjadi @champion!")
```

5. Ringkasan Hasil Eksperimen

Nama Run ID	Algoritma	CV Recall (Mean)	Test Precision	Status Registry
00-dummy-baseline	Dummy Classifier	0.000	0.000	Rejected
01-logreg-scaled	Logistic Regression	0.935	0.962	Approved
10-svc-tuned	**SVC (C=10, rbf)**	**0.965**	**1.000**	**@champion**

6. Kesimpulan

Praktikum ini membuktikan bahwa pencatatan eksperimen berbasis MLflow dan versioning data hash secara drastis menghilangkan subjektivitas, memudahkan perbandingan model secara objektif, serta memastikan model di produksi memiliki *lineage* yang dapat ditelusuri secara utuh.